WEEK 3 – MACHINE LEARNING & AI

Solved Assignments & Mini Project: Unsupervised Learning + Model Evaluation

The uploaded Week 3 sheet focuses on clustering, dimensionality reduction and scaling. Its listed topics are K-Means Clustering, PCA, the Elbow method for choosing K, and model saving/loading with joblib or pickle. ■filecite■turn1file0■L5-L19■

The assignments are: (1) perform K-Means on the Iris dataset and visualize clusters, and (2) apply PCA to reduce dataset dimensions. The Mini Project 3 is “Iris Flower Clustering Project,” with K-Means k=3, a scatter-plot visualization, and comparison of predicted clusters with true labels. ■filecite■turn1file0■L23-L45■

Dataset note: the sheet specifies the Iris Dataset (UCI / Kaggle). For a fully reproducible solution, this PDF uses the standard Iris dataset available through scikit-learn, which has the same four numeric measurements and three species labels.

1. Assignment 1 — K-Means Clustering on Iris Dataset

Goal: divide the Iris observations into 3 groups without using the species labels during training. K-Means is unsupervised: it assigns each point to the nearest centroid and repeatedly updates centroids until the assignments stabilize.

Why scaling? K-Means uses distances. Standardization puts features on comparable scales so a feature with a larger numerical range does not dominate the distance calculation.

Complete Python solution:

```
# Assignment 1: K-Means on Iris dataset

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# 1. Load Iris dataset
iris = load_iris()
X = iris.data

# 2. Convert to DataFrame for inspection
df = pd.DataFrame(X, columns=iris.feature_names)
print(df.head())
print("\nShape:", df.shape)

# 3. Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Apply K-Means with k = 3
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

clusters = kmeans.fit_predict(X_scaled)

# 5. Add predicted cluster to DataFrame
df["Cluster"] = clusters

print("\nCluster counts:")
print(df["Cluster"].value_counts().sort_index())

# 6. Visualize clusters using the first two standardized features
plt.figure(figsize=(8, 6))
plt.scatter(
    X_scaled[:, 0],
    X_scaled[:, 1],
    c=clusters,
    alpha=0.75
)

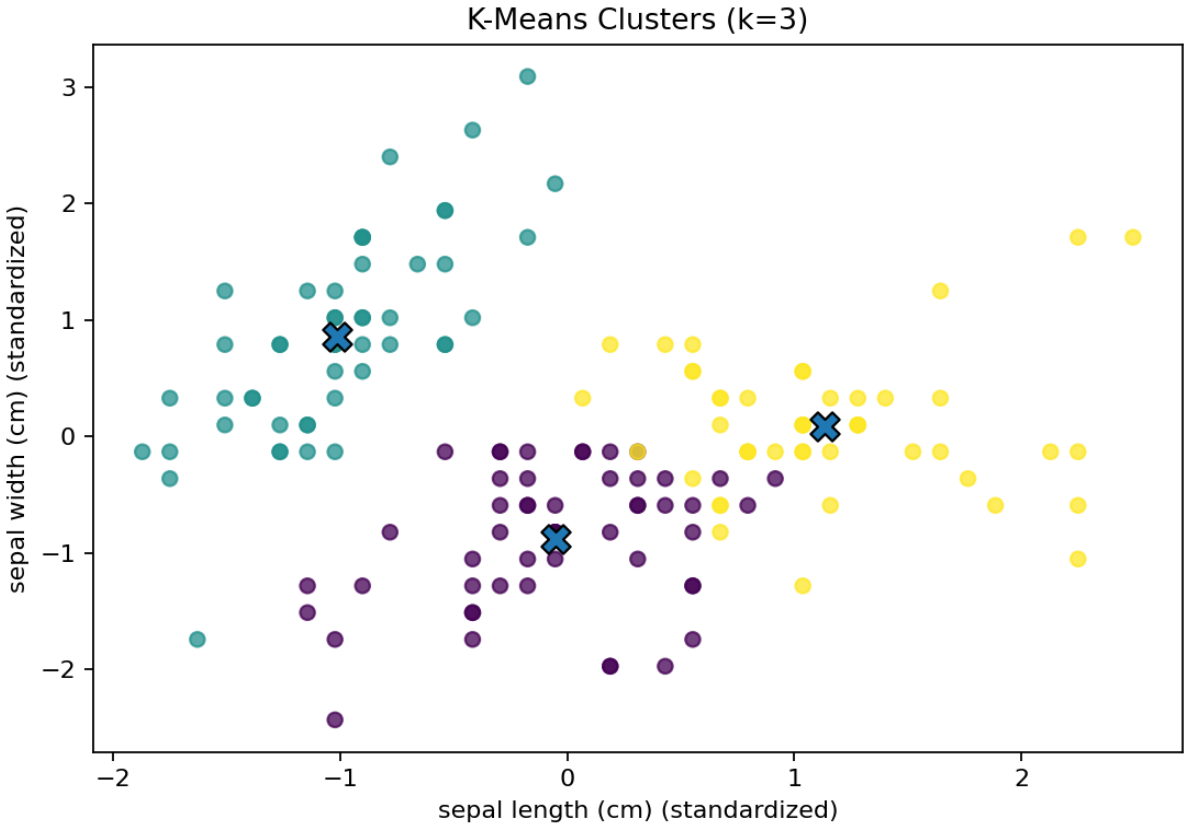
# Plot cluster centers
centers = kmeans.cluster_centers_
plt.scatter(
    centers[:, 0],
    centers[:, 1],
```

```
    marker="X",
    s=150,
    edgecolors="black",
    label="Centroids"
)

plt.xlabel("Sepal length (standardized)")
plt.ylabel("Sepal width (standardized)")
plt.title("Iris K-Means Clustering (k=3)")
plt.legend()
plt.show()
```

Result from the standard Iris dataset:

The model creates 3 clusters. Because K-Means cluster IDs are arbitrary (for example, cluster 0 does not necessarily mean Iris-setosa), the raw cluster numbers should not be interpreted as class names. Using the true Iris labels only for evaluation, the Adjusted Rand Index (ARI) is **0.6201** for this run. ARI is useful because it evaluates grouping similarity while ignoring the arbitrary numbering of cluster IDs.



2. Assignment 2 — PCA for Dimensionality Reduction

Goal: reduce the four-dimensional Iris feature space to two principal components while preserving as much variance as possible. PCA creates new orthogonal directions (principal components) ordered by the amount of variance they explain.

Complete Python solution:

```
# Assignment 2: PCA on Iris dataset

import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. Load data
iris = load_iris()
X = iris.data
y = iris.target

# 2. Standardize before PCA
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Reduce 4 dimensions to 2
pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)

# 4. Display dimensions
print("Original shape:", X.shape)
print("Reduced shape:", X_pca.shape)

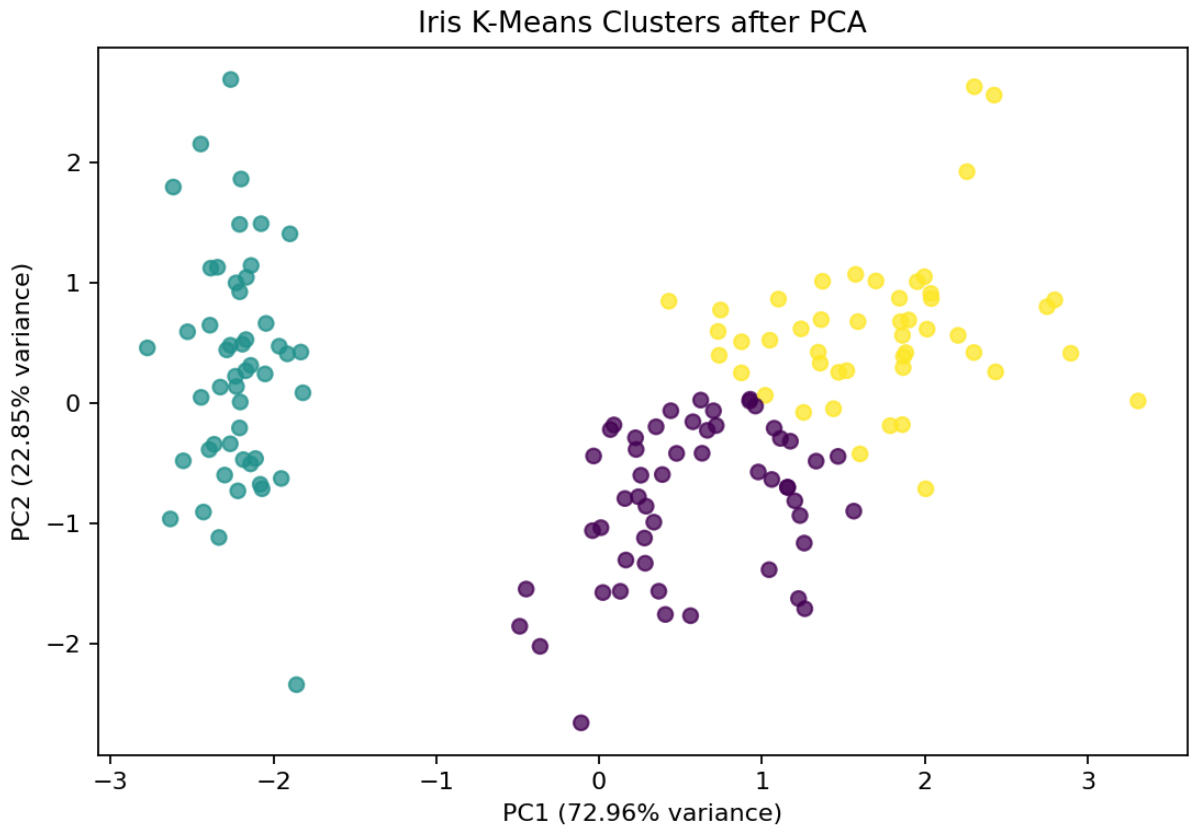
# 5. Variance explained
print("Explained variance ratio:",
      pca.explained_variance_ratio_)
print("Total variance explained:",
      pca.explained_variance_ratio_.sum())

# 6. Visualize the two principal components
plt.figure(figsize=(8, 6))
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=y,
    alpha=0.75
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Iris Dataset after PCA")
plt.show()
```

Result:

After standardization, PCA reduces the original 4 features to 2 features. For the standard Iris dataset, PC1 explains approximately **72.96%** of the variance and PC2 explains approximately **22.85%**. Together they explain **95.81%** of the variance.



3. Mini Project 3 — Iris Flower Clustering Project

The uploaded assignment defines this mini project as Iris Flower Clustering using the Iris Dataset (UCI / Kaggle). The required tasks are K-Means with k=3, cluster visualization and comparison of predicted clusters with true labels. ■filecite■turn1file0■L29-L45■

Complete mini-project code:

```
# Mini Project 3: Iris Flower Clustering Project

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import adjusted_rand_score, confusion_matrix

# ------------------------------------------------------------
# 1. Load Iris dataset
# ------------------------------------------------------------
iris = load_iris()

X = iris.data
y_true = iris.target

print("Features:", iris.feature_names)
print("Target classes:", iris.target_names)
print("Dataset shape:", X.shape)

# ------------------------------------------------------------
# 2. Scale the features
# ------------------------------------------------------------
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ------------------------------------------------------------
# 3. Apply K-Means (k=3)
# ------------------------------------------------------------
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

y_cluster = kmeans.fit_predict(X_scaled)

# ------------------------------------------------------------
# 4. Compare clusters with true labels
# ------------------------------------------------------------
ari = adjusted_rand_score(y_true, y_cluster)

print("\nAdjusted Rand Index:", ari)

print("\nConfusion matrix")
print(confusion_matrix(y_true, y_cluster))

# NOTE:
# Cluster labels are arbitrary. For example, predicted cluster 0
# can correspond to any one of the actual Iris species.

# ------------------------------------------------------------
# 5. PCA to 2 dimensions for visualization
# ------------------------------------------------------------
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# ------------------------------------------------------------
# 6. Visualize K-Means clusters in PCA space
# ------------------------------------------------------------
plt.figure(figsize=(8, 6))
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=y_cluster,
    alpha=0.75
)

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Iris Flower Clustering using K-Means")
plt.show()
```

4. Elbow Method for Choosing K

The Elbow method runs K-Means for several values of K and records the within-cluster sum of squares, often called inertia. As K increases, inertia decreases. The preferred K is often near the point where the improvement starts becoming much smaller—the 'elbow.'

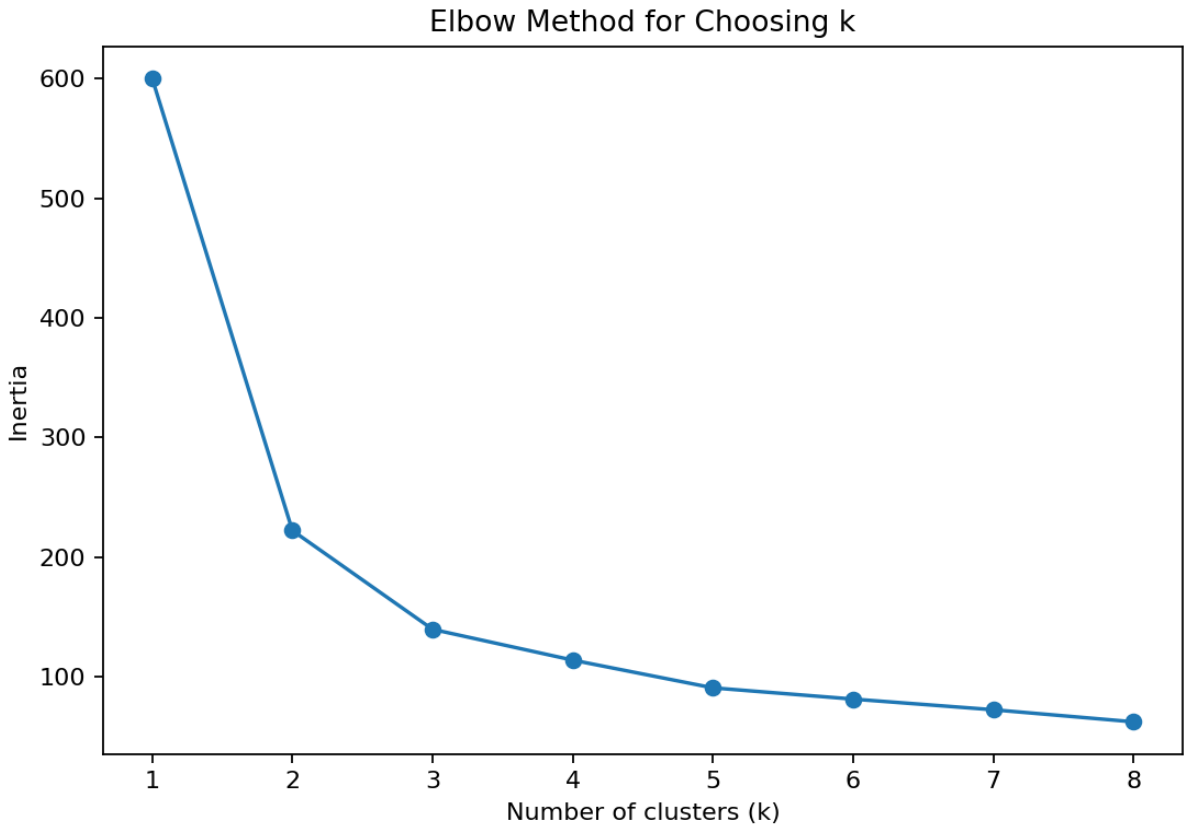
Complete code:

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertias = []

for k in range(1, 9):
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
    model.fit(X_scaled)
    inertias.append(model.inertia_)

plt.figure(figsize=(8, 6))
plt.plot(range(1, 9), inertias, marker="o")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.xticks(range(1, 9))
plt.show()
```



5. Model Saving and Loading

The Week 3 sheet explicitly includes model saving/loading using joblib and pickle. ■filecite■turn1file0■L13-L19■ For scikit-learn models, joblib is commonly convenient for persisting fitted estimators.

Using joblib:

```
import joblib

# Save fitted model
joblib.dump(kmeans, "iris_kmeans_model.pkl")

# Load model later
loaded_model = joblib.load("iris_kmeans_model.pkl")
```

```
# Predict clusters for new scaled samples
new_clusters = loaded_model.predict(X_scaled[:5])
print(new_clusters)
```

Using pickle:

```
import pickle

# Save
with open("iris_kmeans.pkl", "wb") as file:
    pickle.dump(kmeans, file)

# Load
with open("iris_kmeans.pkl", "rb") as file:
    loaded_model = pickle.load(file)
```

6. Actual Results and Interpretation

Dataset: Standard Iris dataset — 150 samples, 4 numerical features, 3 known species.
K-Means: k=3 as required by the assignment.
ARI: 0.6201. This indicates how closely the discovered grouping corresponds to the known species partition, while accounting for arbitrary cluster numbering.
PCA: 4 dimensions reduced to 2; combined variance retained = 95.81%.
Elbow: the plotted inertia curve provides a visual basis for selecting K; the project itself fixes K=3 as required by the Week 3 task.

7. Installation and Execution

```
pip install numpy pandas matplotlib scikit-learn joblib

# Run your Python file:
python week3_iris_clustering.py
```

8. Viva / Interview Questions

Q1. What is unsupervised learning? Learning patterns from data without using target labels during training.
Q2. What is K-Means? A centroid-based clustering algorithm that partitions observations into K groups.
Q3. Why is scaling important for K-Means? Because K-Means uses distances and differently scaled features can distort those distances.
Q4. Why is K=3 used here? The Week 3 mini-project explicitly requires K-Means with k=3. ■filecite■turn1file0■L38-L40■
Q5. What is PCA? A dimensionality-reduction technique that transforms correlated features into ordered orthogonal principal components.
Q6. What does explained variance ratio mean? The fraction of the dataset's variance captured by a principal component.
Q7. What is the Elbow method? A heuristic that examines how inertia changes as K increases and looks for a bend in the curve.
Q8. Can cluster 0 be called Iris-setosa? Not directly. K-Means cluster IDs are arbitrary; they must be mapped or evaluated appropriately.
Q9. What is inertia? The sum of squared distances of samples to their assigned cluster centroids.
Q10. Why use PCA for visualization? It can compress high-dimensional data into two dimensions so patterns and clusters can be plotted.

9. Conclusion

The Week 3 tasks are completed through a reproducible unsupervised-learning workflow: standardize the Iris features, train K-Means with k=3, visualize the discovered clusters, apply PCA for dimensionality reduction, use the Elbow method to study K selection, compare clusters with the known labels using an evaluation measure, and save/load the trained model. This directly follows the topics and Mini Project 3 tasks in the uploaded sheet. ■filecite■turn1file0■L9-L19■ ■filecite■turn1file0■L29-L45■